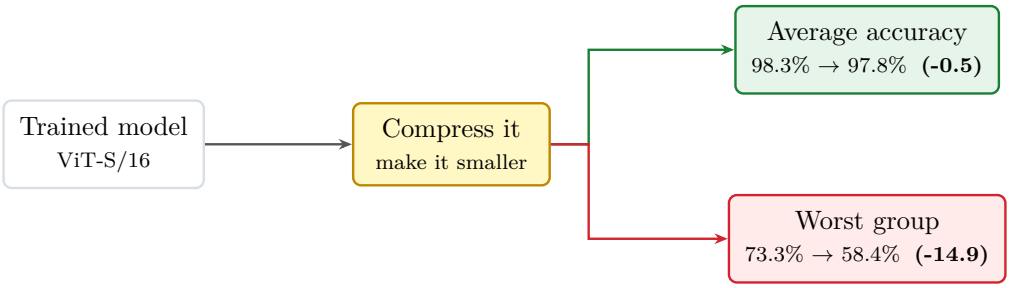
Understanding this project

Compression vs. group robustness on Waterbirds

A plain-English tour of the idea and every file in the repo



29× worse, and the average almost hides it

Everything in here was checked against the actual files in the repo.
Every number is an average over **3 training runs** (seeds 0, 1, 2), from `results/sweep.csv`, `results/sweep_spur*.csv` and `results/retention_*.csv`. Both phases have been run.

Contents

1 The 60-second version **4**

2 The concepts, with zero code **4**
2.1 The dataset: Waterbirds . . . 4
2.2 What the model actually learns . . . 5
2.3 Why average accuracy lies . . . 5
2.4 What "compression" means here . . . 6
2.5 The one rule that makes this a clean experiment . . . 6

3 The pipeline, end to end **7**
3.1 What a ViT block looks like, and where we cut . . . 7

4 Every file, one by one **8**
4.1 `configs/base.yaml` – every number lives here . . . 8
4.2 `src/metrics.py` – what "group" and "worst group" mean . . . 9
4.3 `src/data.py` – loading images, fast . . . 9
4.4 `src/model.py` – building the model and freezing the scope . . . 9
4.5 `src/evaluate.py` – one measurement pass . . . 10
4.6 `src/train.py` – fine-tuning, deliberately naive . . . 10
4.7 `src/compress/` – the four methods, one contract . . . 11
4.8 `src/sweep.py` – the thing that actually runs the experiment . . . 13
4.9 `src/figures.py` – turning the CSV into pictures . . . 14
4.10 Phase 2 files . . . 14
4.11 `tests/` – the safety net . . . 14

5 How to read the figures **15**
5.1 Figure 1 – average vs worst group . . . 15
5.2 Figure 2 – the same average, very different worst group . . . 16
5.3 Figure 3 – which group actually pays . . . 16
5.4 The summary table . . . 16

6 The tricky bits **17**
6.1 Collapse: when the gap narrows and it means nothing . . . 17
6.2 Head removal isn't monotonic, and that's fine . . . 17
6.3 Why the adjusted average, one more time . . . 18
6.4 The resume bug (a real one we found and fixed) . . . 18

7 Phase 2: can we fix it, and does the fix survive? **18**
7.1 The fix: throw away the hardest training images . . . 18
7.2 We got this wrong the first time, and the wrong version is instructive . . . 19
7.3 The measurement: retention ratio . . . 20
7.4 The answers . . . 20
7.5 Why we're 3 points short of the paper . . . 22
7.6 Two things the code refuses to do . . . 22
7.7 One non-obvious engineering bit . . . 22

8 How to actually read this code **22**
8.1 A reading order that works . . . 22
8.2 The exercise that makes it stick . . . 23
8.3 Commands you'll actually use . . . 23
8.4 The three things to remember . . . 23

1. The 60-second version

Here's the whole project in one breath:

> **The question**
> You train a model. It works. Then you shrink it so it's cheaper to run – everyone does this before shipping. **Does shrinking it quietly break the groups it was already bad at?**

And the answer we got: **yes, and if you only look at average accuracy you'd never notice.**

Chop 30% of the weights out of our model and average accuracy drops by **0.5 points**. Sounds free. But the worst-performing group of images drops by **14.9 points**. That's about 29 times bigger. Report only the average and you'd call that compression free, ship it, and the model would be meaningfully worse for exactly the cases it already struggled with.

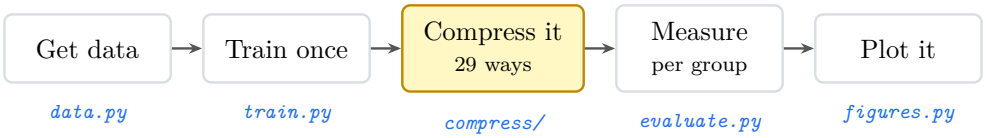
It gets sharper. Delete 6 attention heads out of 72 – that's 2.8% of the weights we're allowed to touch – and the average loses 1.7 points while the worst group loses **18.2**.

And then we asked the obvious follow-up: what if you *fix* the robustness problem first, and only then compress? The short answer is that the fix works (worst group goes from 73.3% to **87.9%**), it *does* survive gentle compression, and it stops surviving at exactly the point where compression has broken the model for everyone. Section 7 has the numbers.

The three words you need

- **Shortcut.** Our dataset is birds. Water birds are usually photographed on water, land birds on land. So the model learns "water background = water bird" instead of actually learning what a bird looks like. It's lazy, and it works most of the time.
- **Group.** Split the test set by (what bird it is) × (what background it's on). Four groups. Two of them agree with the shortcut, two fight it. The ones that fight it are where the model is weak.
- **Worst-group accuracy.** Accuracy on the single worst of those four groups. This is the number that actually tells you whether the model learned birds or learned backgrounds.

What the repo does, start to finish



That's Phase 1. Phase 2 adds one step in front of it – throw away some training images first, then do the same thing – and asks whether the fix survives. Both phases are done. Section 7 has the answers, Section 5 explains how to read the pictures.

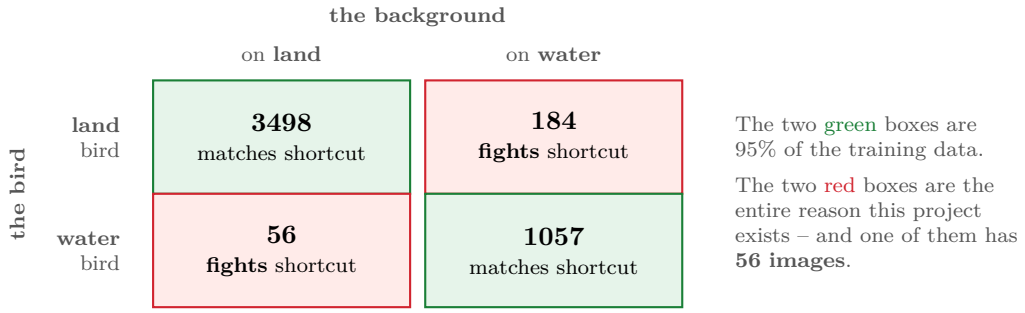
2. The concepts, with zero code

Skip this part if you already get shortcut learning. Otherwise this is the foundation for everything else.

2.1 The dataset: Waterbirds

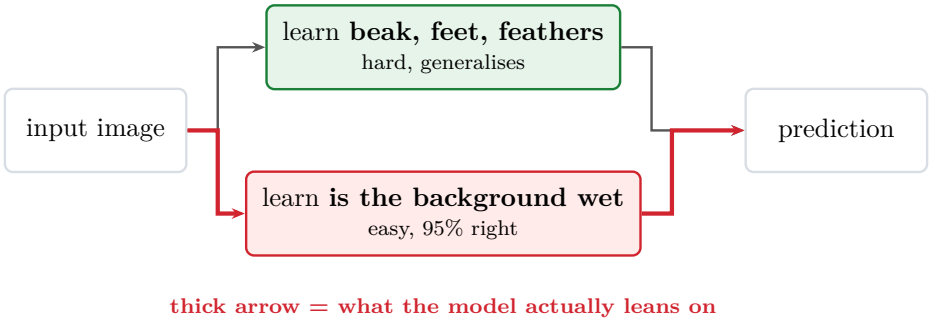
Waterbirds is a dataset built on purpose to be a trap. Someone took photos of birds, cut the birds out, and pasted them onto new backgrounds. They controlled exactly how often each bird type lands on each background type.

Here's what the training set looks like. Each box is a group, and the number is how many training images are in it:



Look at that imbalance. 3498 land birds on land, but only 56 water birds on land. If you're a model trying to minimise your loss, you will notice that "is the background wet?" predicts the label correctly 95% of the time, and that's a much easier thing to learn than bird anatomy.

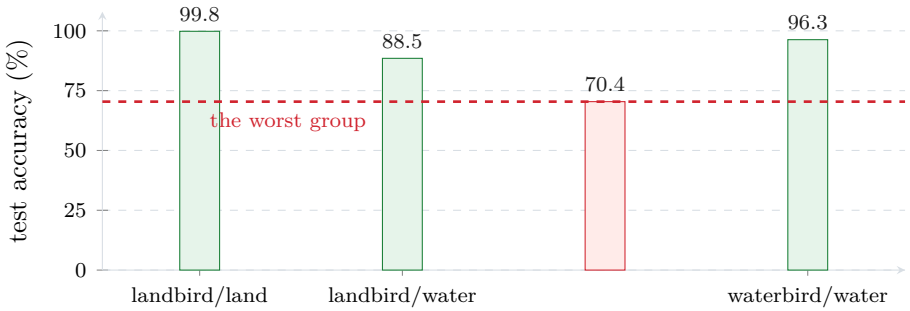
2.2 What the model actually learns



Nothing in plain training punishes the shortcut. It's not a bug in the model, it's the rational move given the data. That's what makes it interesting.

2.3 Why average accuracy lies

Average accuracy lies to you. Our model scores 98.2% and you'd think you're done. But break it down by group:



There it is. **99.8%** on the easy group, **70.4%** on water birds photographed on land – the group with 56 training images that fights the shortcut. A 29-point spread, and the average buries it.

So we track three numbers, and you need to know the difference:

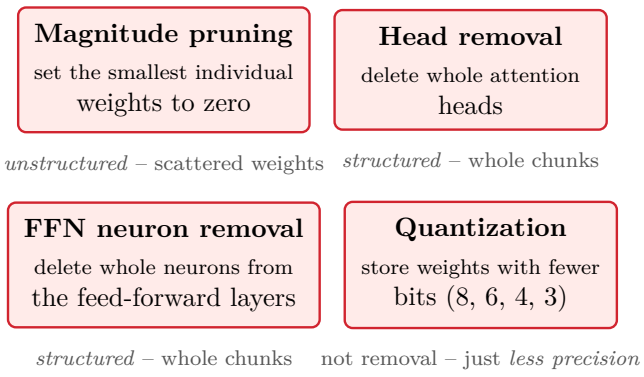
Number	On our model	What it means
plain average	91.7%	straight mean over the test set
adjusted average	98.2%	reweighted to match *training* group frequencies
worst group	**70.4%**	the single weakest group. the honest one

Why two different "averages"?

Not because the test set has equal numbers of each bird – it doesn't, and neither split does. Both are about 77% land birds. What the test set balances is the *background*. Land birds are 2255 on land and 2255 on water; water birds are 642 and 642. So the background predicts the label 95% of the time in training and exactly 50% at test time. That is the whole point of the construction: at test time the shortcut is worth nothing. The consequence is that the two groups that fight the shortcut are 5% of training but 50% of the test set. A plain test average hands them ten times the weight training ever gave them. The *adjusted* average reweights per-group accuracy by training frequency instead, and that's the ∼98% number everyone in this research area quotes. We report it so our numbers are comparable to published work. The code is `metrics.py`, function `adjusted_average`.

2.4 What "compression" means here

Compression means making the model smaller or cheaper after it's already trained. We try four flavours, and they're genuinely different operations:



Each one has a knob we turn. Prune 30%, 50%, 70%... remove 6 heads, 12 heads, 24... quantize to 8 bits, 6, 4, 3. 29 settings total. For every single one we re-measure all four groups.

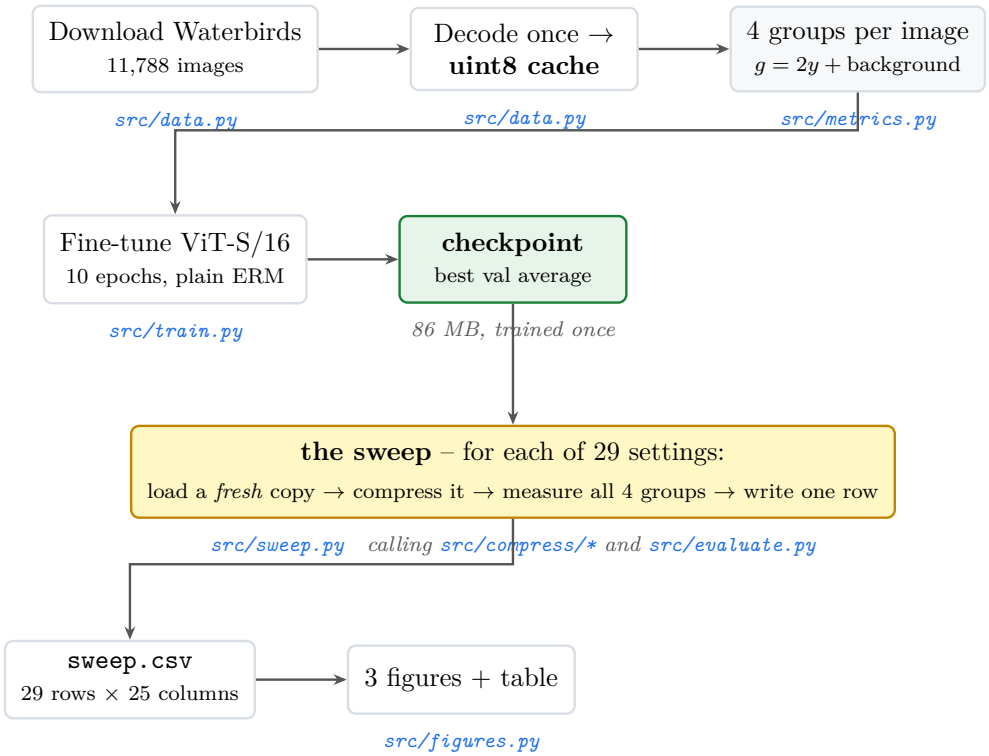
2.5 The one rule that makes this a clean experiment

We never retrain after compressing

This is the whole design. Compress the finished model, measure immediately, done. If we fine-tuned afterwards the model would just re-learn the shortcut and we'd be answering a completely different question. Same reason training is deliberately *group-blind*: no group labels, no reweighting, no balancing. The model is *supposed* to learn the shortcut. If we fixed that during training, worst-group accuracy would start high and there'd be nothing to measure.

3. The pipeline, end to end

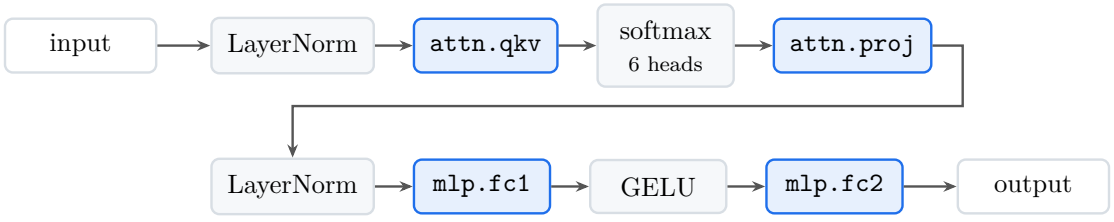
Now the code. Before looking at any single file, get this picture in your head – everything else is a detail hanging off it.



Read that top to bottom and you've got the shape of the repo. Now the important bit about *where* in the model the compression actually happens.

3.1 What a ViT block looks like, and where we cut

Our model is ViT-S/16: 12 identical blocks stacked on top of each other. Each block has two halves – an attention part and a feed-forward (MLP) part. Here's one block, with the four weight matrices we're allowed to touch highlighted:



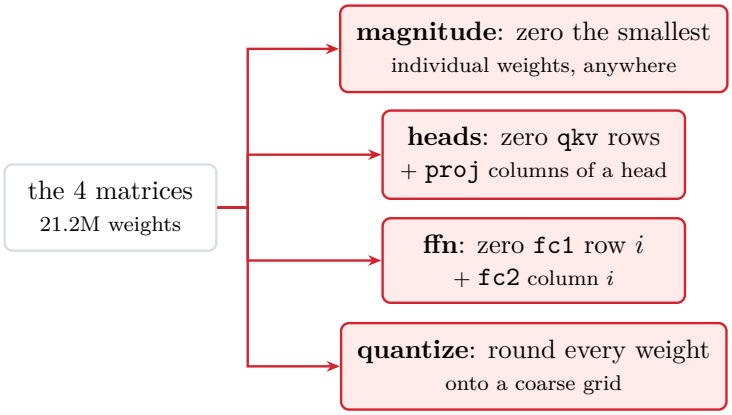
The four blue boxes are the only things any compression method may touch. 12 blocks × 4 matrices = **21,233,664 weights**.

The "scope" and why it's frozen

Everything else – the patch embedding, the position embeddings, the class token, every LayerNorm, every bias, and the final classifier head – is **off limits**, for every method, always. That's the list in `model.py` called `TARGET_SUFFIXES`.

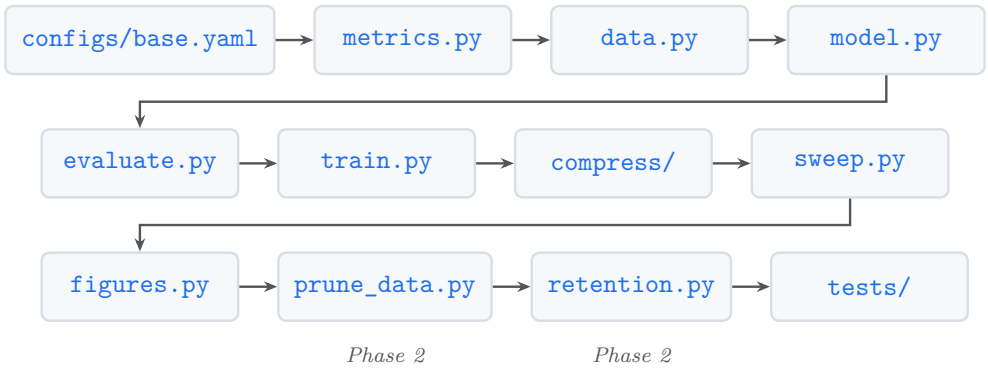
Why bother? Because "we pruned 30%" has to mean the same thing in all four experiments. If head removal were allowed to touch LayerNorms and magnitude pruning wasn't, the sparsity numbers wouldn't be comparable and the whole results table would be meaningless. One scope, decided once, reused everywhere.

And here's where each method attacks that scope:



4. Every file, one by one

Read them in this order – each one only depends on the ones above it.



4.1 `configs/base.yaml` – every number lives here

What it does: holds every constant the pipeline depends on. Learning rate, epochs, which sparsity levels to sweep, where files go.

Why it exists: so no number is hardcoded in two places. If you want to sweep different sparsity levels, you edit this file and nothing else.

Look at: the `sweep:` block – those lists are literally the 29 experiments. `magnitude_sparsity` has 7 values, `heads_removed` 7, `ffn_fraction` 7, and quantize is 4 bit-widths $\times$ 2 granularities $= 8$. That's $7 + 7 + 7 + 8 = 29$.

4.2 `src/metrics.py` – what "group" and "worst group" mean

What it does: turns predictions into group-aware accuracy numbers. No model code, no PyTorch layers – just counting.

The one function to understand:



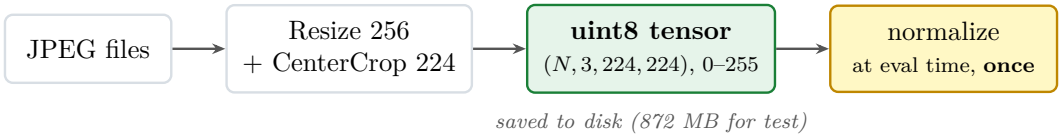
The tricky bit: `adjusted_average`. It does *not* compute a plain mean. It takes per-group accuracy and reweights by how common each group was **in training** (72.9%, 3.8%, 1.2%, 22.0%). That's why our headline is 98.2% and not 91.7%. Both are correct; they answer different questions.

Also note: `TRAIN_GROUP_COUNTS = [3498, 184, 56, 1057]` is asserted against the real data by a test. If that assert ever fails, group indexing is wrong and every number downstream is garbage. It's a tripwire.

4.3 `src/data.py` – loading images, fast

What it does: downloads Waterbirds, reads `metadata.csv`, exposes the three official splits, and builds the uint8 caches.

Why the cache exists: we evaluate the test set about 30 times during a sweep. Decoding 5,794 JPEGs each time would dominate everything. So we decode once, resize once, and keep raw pixels in a big tensor.



The double-normalize trap

Notice the cache stores **raw 0–255 pixels**, not normalized floats. That's deliberate and it's the single easiest way to wreck this project.

If you normalized before caching, and then the eval code normalized again (which it does, every time), every image would be silently wrong. Your accuracy would drop a bit, nothing would crash, and you'd spend a week blaming compression. The comment in the file calls this "the double-normalize trap". Caching pre-normalization is what avoids it.

Look at: `normalize_uint8()` – it's three lines and it's the only place normalization ever happens.

4.4 `src/model.py` – building the model and freezing the scope

What it does: creates the ViT, loads checkpoints, and defines exactly which weights compression may touch.

Two functions that matter:

- `targeted_named_weights(model)` – returns the list of **21,233,664** weights in scope. Every compression method starts by calling this. One definition, four methods.
- `fresh_model(ckpt, device)` – loads a brand new, clean, untouched model from the checkpoint file. The sweep calls this so that every configuration starts from the same pristine weights.

Also: `assert_structure()` checks the model really is 12 blocks × 6 heads × 64 dims, that `attn.norm` is an `Identity`, and that the matrices have the shapes we expect. This isn't paranoia – the head-removal maths in Section 6 is only correct *because* of that layout. If a future version of `timm` changes it, this assert fires instead of silently producing wrong results.

4.5 `src/evaluate.py` – one measurement pass

What it does: runs the model over a cached split and returns the full metric dict. It's 50 lines and you can read it in a minute.

The one line to notice: `model.eval()` at the top. Without it, dropout and drop-path stay active and inject random noise – which would look exactly like a compression effect. The comment says so.

Also returns `pred_mean`, the average predicted class. This is the collapse detector; see Section 6.

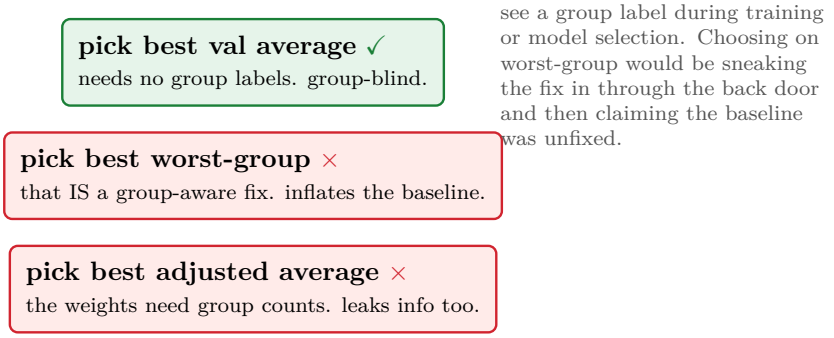
4.6 `src/train.py` – fine-tuning, deliberately naive

What it does: takes a model pretrained on ImageNet, swaps in a 2-class head, and fine-tunes on Waterbirds for 10 epochs. AdamW, learning rate 1e-5, cosine schedule, batch size 32.

What it deliberately does NOT do: no group labels, no reweighted loss, no balanced sampler, no GroupDRO, nothing clever. Plain shuffling, plain cross-entropy.

> **"Why would you train it badly on purpose?"**
> Because the model is *supposed* to learn the shortcut. That's the thing we want to study. If we trained it group-balanced, worst-group accuracy would start around 90%, there'd be no gap, and compression would have nothing interesting to break. The naive model is the subject of the experiment, not a mistake.

The subtle choice – which epoch do we keep? We save the checkpoint with the best **plain validation average** accuracy. Not worst-group. Here's why that matters:



What Phase 2 added to this file: `prune_indices` to train on a subset, `score_epoch` to record per-sample difficulty, and `stop_after_epoch` to score a model without overwriting an existing checkpoint. Section 7 explains what those are for.

Three more flags exist only because a long unattended run went wrong, and they are worth knowing about if you ever leave this training overnight:

- `-resume` picks a killed run back up from where it stopped, putting the optimiser and the learning-rate schedule back exactly as they were. Without that last part you'd restart mid-schedule and get a model that looks fine and isn't.

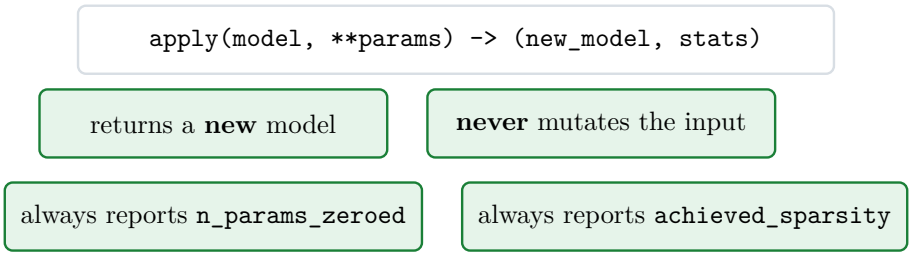
- `-allow-overwrite` is a refusal, not a feature. A fresh run will not start on top of a model that already exists unless you say this. Saved models are not backed up anywhere and cost half an hour each to rebuild.
- `-num-workers` loads images on several threads. It is set to 0, which sounds wrong but was measured: it made no difference to speed here, and 0 reproduces Phase 1's results exactly, so 0 it is.

The bug these came from

`stop_after_epoch` exists to score a model *without* touching the saved copy. Its check was one line too late, so a scoring run reached the "save if this is the best so far" step first and overwrote the Phase 1 model with a much worse, half-trained one. The flag whose only job was to protect that file is what destroyed it.
Nothing noticed – the replacement looked like a perfectly normal model. It had to be retrained from scratch, which worked only because training here turns out to be exactly repeatable: all ten epochs came back identical to the original. Gate V13 now checks this specific case, and `preflight.py` re-checks every saved model against the record of how it was trained before any long run starts.

4.7 `src/compress/` – the four methods, one contract

Every method, without exception, follows the same shape:



That contract is enforced by tests, not by good intentions. The shared helper `clone_model()` in `_common.py` is a deep copy – that's where "never mutates" starts.

Masking, not shrinking

Two of the methods (heads, ffn) delete whole structural chunks. You might expect them to physically make the tensors smaller. They don't – they set things to zero and leave the shapes alone.
Why: so `achieved_sparsity` means the same thing for all four methods. If head removal physically narrowed `qkv`, then "fraction of targeted weights zeroed" would be measuring something different than it does under magnitude pruning, and you couldn't put them on the same axis. The tests prove the masked version gives *bit-identical* outputs to really deleting the parts.

`magnitude.py` – zero the smallest weights

Take all 21.2M weights, sort by absolute value, zero the smallest k of them. Global ranking: a tiny weight in block 11 gets pruned before a big weight in block 0.

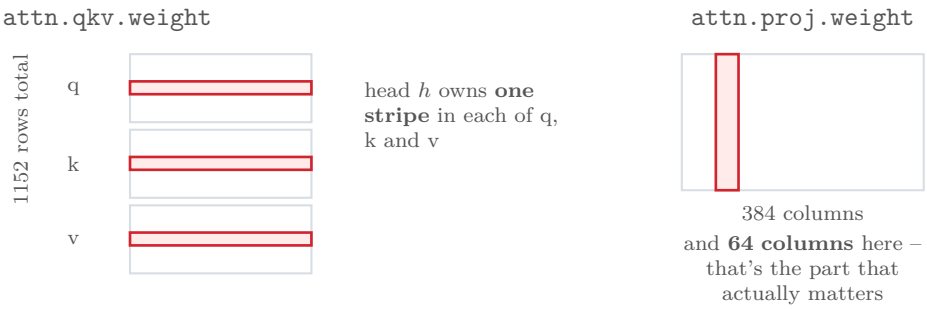


The tricky bit – ties. With 21 million float32 numbers, the threshold value is *not unique*. Several weights have exactly the same magnitude (measured: ties run 1–3 deep). If you wrote the obvious mask `w.abs() > threshold`, you'd zero *every* tied weight and overshoot your target. You'd ask for 30% and get 30.0001%.

So the code zeroes everything strictly below the threshold, counts how many that was, and then spends an explicit "tie budget" on exactly as many tied weights as it still needs – in a fixed order so it's reproducible. Result: exactly k weights, every time. That's why the test is an equality and not a "close enough".

`heads.py` – delete whole attention heads

Each block has 6 attention heads. A head owns a specific slice of two matrices:



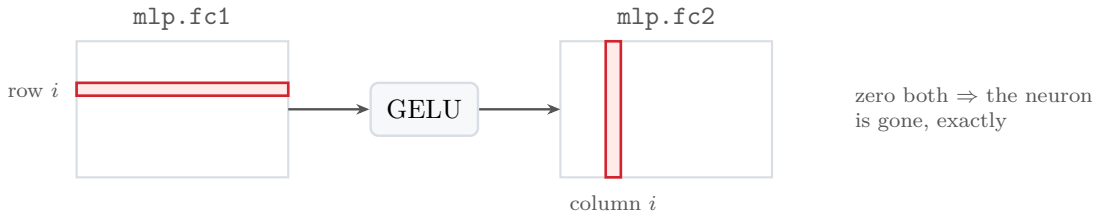
Why zeroing those columns genuinely deletes the head. At the end of attention, the 6 heads' outputs are laid side by side into one long vector, and then `proj` maps it back down. Head h's output lands in columns $[h \cdot 64, (h{+}1) \cdot 64)$ of that vector. Zero those columns of `proj` and head h contributes **exactly nothing** to the output. Not approximately – exactly.

The `qkv` rows get zeroed too. They can't change the output anymore (proj already blocks it), but they *are* dead weight, and leaving them out of the count would understate how much of the model a head sweep removes.

One guard rail: at least 1 head must survive per block. Empty a block's attention completely and it stops being a compressed transformer and becomes a different architecture – and the accuracy cliff would get blamed on "head sparsity". 12 blocks $\times$ 5 removable $= 60$ max, which is exactly where the sweep stops.

`ffn.py` – delete whole MLP neurons

Simplest one conceptually. A "neuron" in the feed-forward layer is a matched pair: row i of `fc1` (which computes it) and column i of `fc2` (the only way it reaches the output).



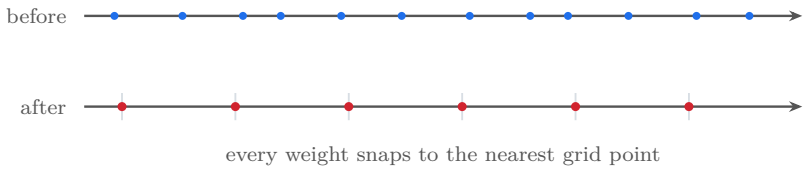
The "wait, what about the bias?" question. `fc1.bias` is *not* zeroed – biases are outside the frozen scope. So a pruned neuron still outputs a constant `GELU(bias_i)`. Doesn't that break exactness?

No. That constant gets multiplied by `fc2`'s zeroed column, so it contributes exactly zero. Removal stays exact without widening the scope. Nice little result, and there's a test that checks it against a reference model.

No per-block floor here, unlike heads. Emptying one block's MLP leaves attention and the residual stream intact – it degrades the model rather than changing its architecture. So blocks are free to give up wildly different numbers of neurons, and where the ranking chooses to cut is itself a result.

`quantize.py` – fewer bits per weight

Instead of removing weights, store each one with less precision. Round every weight onto an evenly spaced grid.



Two knobs, and one matters more than you'd think:

- **bits** – 8, 6, 4, or 3. Fewer bits = coarser grid.
- **granularity** – `per_tensor` uses one grid spacing for the whole matrix; `per_channel` uses a separate one for each output row.

At 4 bits, per-channel keeps **60.6%** worst-group accuracy and per-tensor keeps **23.8%**. Same bit budget, wildly different outcome. One big outlier weight ruins the grid spacing for the entire tensor, and per-channel contains the damage to one row.

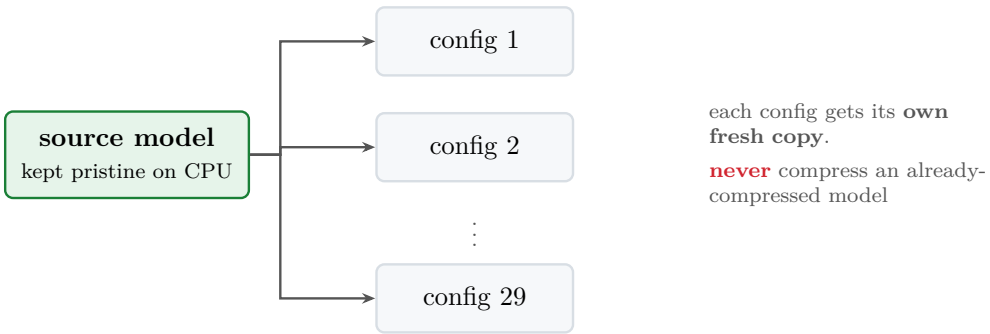
Note: this is *simulated* quantization – quantize then dequantize back to float32. We're measuring accuracy, not speed, and the numerical effect is reproduced exactly. Real int4 kernels are CUDA-only and wouldn't change a single accuracy number.

Tiny detail worth knowing: at `bits=32` the code short-circuits and returns the model untouched. If it actually ran the maths, dividing by a scale that small would push values past what float32's mantissa can hold, and the "identity" would be off by about 1e-7 – failing a test that demands exact identity.

4.8 `src/sweep.py` – the thing that actually runs the experiment

What it does: loops over all 29 configurations. For each one: load a fresh model, compress it, evaluate it, write a row to `results/sweep.csv`.

This is the most important file in the repo and it exists mostly to protect two invariants.



The most dangerous bug available in this project

If config 2 accidentally compressed the output of config 1, the damage would compound. And here's the evil part: **the output would still look completely plausible.** You'd get smooth, believable curves that are entirely wrong, and nothing would crash or look weird.

So the code doesn't just trust the control flow. Before every single config it recomputes a `_fingerprint()` of the source model – the count of zeros plus the sum of all weights – and asserts it hasn't changed. If any compression method ever broke its "don't mutate the input" promise, that assert fires immediately instead of 29 rows later.

Invariant 2: resumability. Each row is written and flushed to disk the moment it's done. A sweep is ∼30 full passes over 5,794 images; if your laptop sleeps halfway through, `-resume` picks up from the last finished row instead of starting over.

Look at: `run_one()` – it's about 25 lines and it's the entire experiment in miniature: compress, evaluate, build a row.

4.9 `src/figures.py` – turning the CSV into pictures

What it does: reads `sweep.csv` plus `baseline.json` and writes the three figures and the summary table.

The one concept: collapsed configurations are drawn as *hollow* markers and excluded from the trade-off plot. Section 6 explains why that matters so much.

4.10 Phase 2 files

`src/prune_data.py` – scores every training image by how "hard" it is, then removes the hardest ones and writes a list of which images to keep. It holds *two* rules, chosen with `-variant`: `global` deletes the hardest images anywhere (the version that failed), and `spurious` deletes only from the groups where the background agrees with the bird (the version that works). See Section 7.

`src/retention.py` – compares the ERM model against the pruned model at matched compression and computes the retention ratio. This is Phase 2's headline number.

`src/migrate_csv.py` – a one-shot utility that adds the new `tag` column to the Phase 1 CSV so both phases share one schema. You'll never run it again.

`run_phase2.sh` – runs the whole Phase 2 pipeline in stages. Every step skips if its output already exists, so an interrupted night resumes by just running it again. Two stages exist only because an overnight run went wrong once: **stage 0** checks everything up front – disk space, the image caches, and whether each saved model still matches the record of how it was trained – and refuses to start if anything is off. **Stage 5** checks afterwards whether every piece of work really finished, by looking at the files rather than the log.

`src/preflight.py` and `src/verify.py` are those two checks. `verify.py` is the one to run if you ever want a straight answer to "did it all actually finish?" – it reads the output files and says yes or no. The log is not a reliable guide: on one run, 12 steps reported failure and 11 of them had in fact completed their work and only tripped on the way out, when the disk filled up.

`src/progress.py` – prints the progress bar you can watch during a long run.

4.11 `tests/` – the safety net

64 tests, plus 15 more that drive the run script itself. They're not there for code coverage points; each one pins down a specific claim the rest of the code makes. The source comments literally refer to them by number ("V4 is an equality check").

Gate	Claims	Why you'd care
V1	group counts are `[3498,184,56,1057]`	group indexing is right
V3	quantize at 32 bits is identity	the method has no hidden bias
V4	magnitude zeroes *exactly* k	the tie handling works
V5	masking = real deletion, bit for bit	structured removal is exact
V7b	the baseline hasn't collapsed	accuracy isn't a mirage
V8	`apply` never mutates its input	the sweep is trustworthy
V10	pruned set is class-balanced	Phase 2 removal is fair
V12	resume key separates seeds	a 2nd seed doesn't silently skip
V13	a cut-short run saves no model	it once destroyed the Phase 1 model
V14	Python and the shell agree on a name	else half-trained models look done
V15	the run script's skip/resume logic	an interrupted night resumes right
V17	the fix never deletes a weak-group image	the whole point of the fix
V18	the fix evens up the two classes	matches what the paper describes

Run them with `pytest tests/ -q`. Takes about 35 seconds.

Why there are suddenly more of these

V13–V18 all exist because something went wrong on a real overnight run. V13 is the sharpest: the flag whose entire job was to stop the Phase 1 model being overwritten had its guard in the wrong place, and overwrote it. The model had to be retrained from scratch to get it back. Each of those gates is one specific mistake that can no longer repeat silently.

5. How to read the figures

Running `figures.py` gives you three pictures and one table per model. They all follow the same handful of rules, so once you can read one you can read all of them. This section is those rules.

The single most important convention

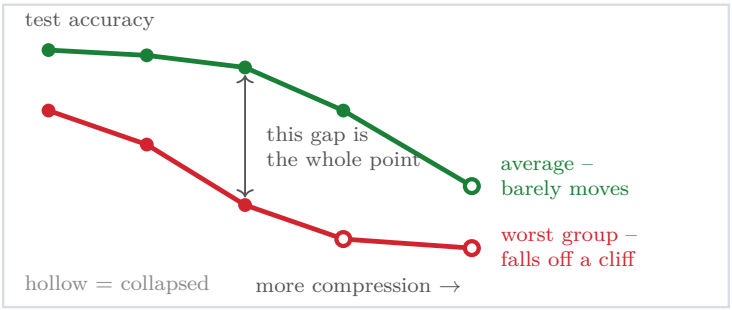
A **filled dot** is a real measurement. A **hollow dot** is a model that has stopped working – it has given up and started answering almost every image the same way. Hollow dots are drawn so the line still connects, but **you must not read anything into them**. Section 6 explains why a broken model can look like it has a small gap.

5.1 Figure 1 – average vs worst group

Five small plots, one per compression method. In each one:

- **Left to right** always means *more compression*. For the quantization plots that means the number of bits *decreases* left to right (8, 6, 4, 3), because fewer bits is more squeezing. The axis is deliberately flipped so that "worse" is always rightward in every panel.
- **Up and down** is test accuracy, always on the same 0–100% scale so the five panels are directly comparable.
- **Two lines.** The upper one is average accuracy – what a normal evaluation would report. The lower one is worst-group accuracy.

What you are looking for: the vertical distance between the two lines, and how fast it opens up. That gap is the whole finding. If the two lines fell together, compression would be fair. They don't.



5.2 Figure 2 – the same average, very different worst group

One scatter plot. Each dot is one of the 29 compression settings. Left–right is average accuracy, up–down is worst-group accuracy.

How to read it: pick a spot on the horizontal axis and look up. You will find several dots stacked vertically. Those are compression settings that a standard evaluation would call *equally good* – same average accuracy – sitting at wildly different worst-group accuracies. That vertical spread is the information the average throws away. The further a dot sits below the others at its own horizontal position, the more that particular setting hurt the group that was already weakest.

5.3 Figure 3 – which group actually pays

Five panels again, same axes as Figure 1, but now **four lines, one per group**. The four groups are the two bird types crossed with the two backgrounds.

How to read it: two of the lines stay high and two dive. The two that dive are the groups where the background disagrees with the bird – a land bird photographed over water, a water bird photographed on land. Those are the images where the background shortcut gives the wrong answer, so they are the ones that need the model to have actually learned birds. Figure 3 is the picture of *who* pays for compression, where Figure 1 only says *that* someone does.

5.4 The summary table

`summary_table.csv` is the same information as Figure 1 in numbers, one row per compression setting. The columns worth reading first:

`achieved_sparsity`	how much was really removed, not how much we asked for
`acc_adj_avg`	the average accuracy a normal evaluation reports
`acc_wg`	worst-group accuracy
`gap_adj`	the two subtracted – the number this project is about
`pred_mean`	the collapse detector. Near 0 or near 1 means broken

If you only ever look at two columns, look at `acc_wg` and `pred_mean`: the first is the result, the second tells you whether to believe it.

A trap worth naming

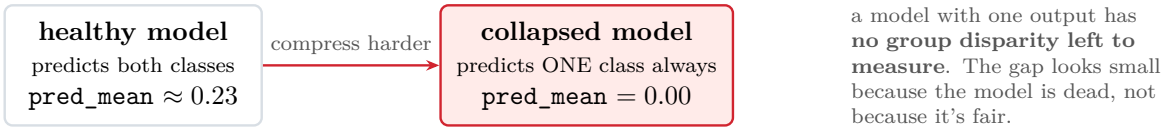
Some rows have a *small* gap and look reassuring. Check `pred_mean` before believing any of them. A model that answers "land bird" to every single image has no gap at all between its groups, because it is equally useless on all of them. A tiny gap can mean "fair" or it can mean "dead", and only `pred_mean` tells you which.

6. The tricky bits

This is the part that separates "I read the code" from "I understand the code". None of these are obvious, and most of them exist because someone would otherwise get a wrong answer that looks right.

6.1 Collapse: when the gap narrows and it means nothing

Look at the results past about 80% sparsity and something weird happens – the gap between average and worst-group accuracy starts *shrinking* again. Robustness returning? No. The opposite.



Here's the real row from our data, at 80% magnitude sparsity:

	g0	g1	g2	g3	adj avg
accuracy	100%	100%	0%	0%	**76.8%**

That model predicts "landbird" for every single image. It gets 100% on both landbird groups and 0% on both waterbird groups. And it still posts a **76.8% adjusted average**, which would look like a mediocre-but-working model if you only read that one column.

That's why `pred_mean` exists in the CSV. It's the average predicted class; if it's pinned near 0 or 1, the model has collapsed. Those points get drawn hollow in the figures and excluded from the trade-off plot.

6.2 Head removal isn't monotonic, and that's fine

You'd expect: remove more heads, get worse. Our data says removing **48** heads scores *better* on worst-group (34.1%) than removing **36** (1.1%).

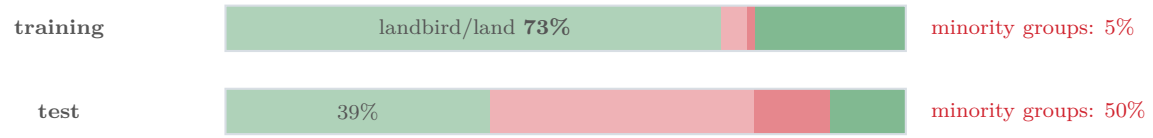
Two reasons, and neither is a bug:

- Ranking heads by weight size doesn't rank them by how much the model actually *uses* them. Cutting the "smallest" 36 can land worse than cutting the smallest 48.
- Near-collapse, which group is "worst" keeps reshuffling. At 36 heads the worst group is waterbird-on-land; the numbers are bouncing around a nearly dead model.

Treat the deep end of the head sweep as noise around a broken model, not as signal. The honest finding lives at the shallow end: 6 heads out of 72 costing 22 points.

6.3 Why the adjusted average, one more time

This trips everyone up, so here it is as a picture:

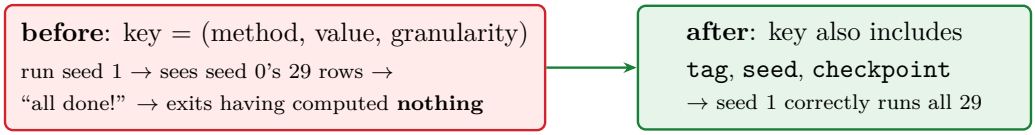


Careful about what "balanced" means here. The test set is not balanced by bird: both splits are about 77% land birds. It is balanced by *background* – 50/50 inside each bird class, where training is 95/5. So the two groups that fight the shortcut are 5% of training and 50% of the test set, and a plain test average gives them ten times the weight they had in training. The adjusted average re-weights back to training proportions, which is the convention in this research area. That's the only reason our headline is 98.2% instead of 91.7% – same model, same predictions, different weighting.

6.4 The resume bug (a real one we found and fixed)

Worth knowing because it's the kind of bug that produces *no error at all*.

The sweep skips configurations already in the CSV. The check used to be "have I seen this (method, value, granularity) before?". Which is fine when there's exactly one model.



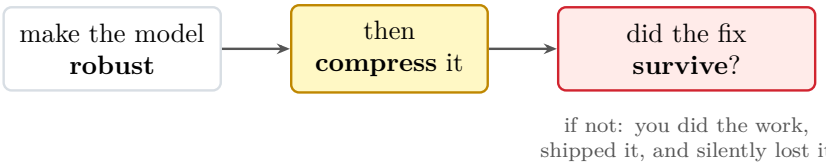
The CSV would have looked complete and you'd have "three seeds" that were actually one seed copied. Gate V12 now pins this down.

7. Phase 2: can we fix it, and does the fix survive?

Status: run, 3 seeds, and it worked

Everything in this part has been executed. Headline: the fix takes worst-group accuracy from **73.3%** to **87.9%**, and that gain survives gentle compression but not heavy compression. Numbers below.

Phase 1 compressed a model that was never made robust in the first place. Fair enough – but a practitioner would say "obviously, just fix the robustness first". Phase 2 does exactly that, and then asks the uncomfortable follow-up.



7.1 The fix: throw away the hardest training images

This is from a 2025 paper (Mulchandani and Kim, ICLR). The idea is counterintuitive and kind of beautiful:

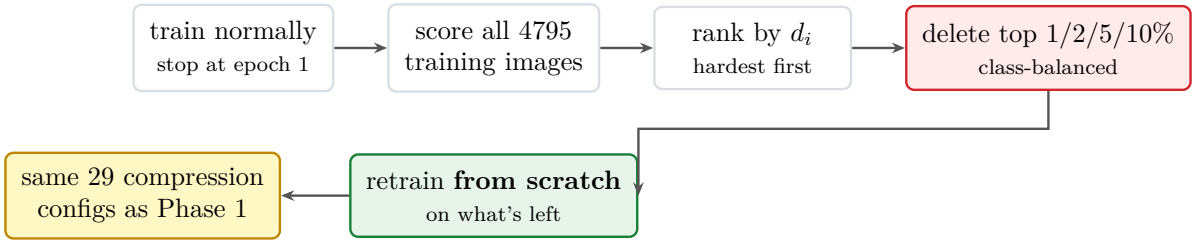
Why deleting data makes a model MORE robust

A model takes the background shortcut because some training images are hard to classify on their actual bird features. Where the real feature is hard, the easy fake feature wins.
So: find the images the model finds hardest, delete them, and retrain. Without those images pushing it toward the shortcut, the model has less reason to learn "wet background = waterbird". You lose a bit of data and gain robustness.

How "hard" is measured. Train normally, stop at an early epoch, and for each training image compute how far the model's predicted probabilities are from the correct answer:

$$d_i \;=\; \| \, p(x_i) - \text{onehot}(y_i) \, \|_2$$

Big d_i means the model is confidently wrong or very unsure – a hard image.



7.2 We got this wrong the first time, and the wrong version is instructive

Our first attempt deleted the hardest images *anywhere* in the training set, without ever looking at which group they belonged to. That felt like the honest thing to do – a fix that needs group labels is a weaker fix. We ran it, and worst-group accuracy got *worse*, steadily worse the more we deleted.

Here is why, and it is obvious in hindsight. Think about which images a model finds hardest. They are exactly the ones where the background lies: the water bird standing on grass, the land bird over a lake. Those images are hard *because* the shortcut fails on them. So "delete the hardest images" quietly means "delete the images that were holding the weak groups up".

We measured it. The worst group has only 56 training images to begin with. At a 10% deletion budget, the group-blind rule threw away **38 of those 56**:

	share of each group deleted			
deletion budget	land/land (3498)	land/water (184)	**water/land (56)**	water/water (1057)
1%	0.3%	7.6%	**14%**	1.5%
2%	0.6%	15%	**23%**	3.3%
5%	1.7%	33%	**45%**	9.0%
10%	4.3%	48%	**68%**	19%

And then we actually read the paper properly, and found it says this itself. For a dataset like Waterbirds, where you *can* tell which images have the misleading background, the paper's method is told the groups and deletes only from the groups the shortcut agrees with. It states plainly that deleting the globally hardest images "is not feasible" here, for precisely the reason our numbers show.

The corrected rule, in one sentence

Only consider images where the background *agrees* with the bird – the easy, shortcut-confirming ones – and delete the hardest of those. Never touch the images where the background disagrees. Those are the ones teaching the model to actually look at the bird.

So the project now has both versions, and keeps both:

- `-variant global` – the group-blind rule. Kept because it demonstrates the failure the paper warns about. Its results live in `old_run_Oct26/`.
- `-variant spurious` – the corrected rule. This is the one that reproduces the paper, and the one all the numbers below come from.

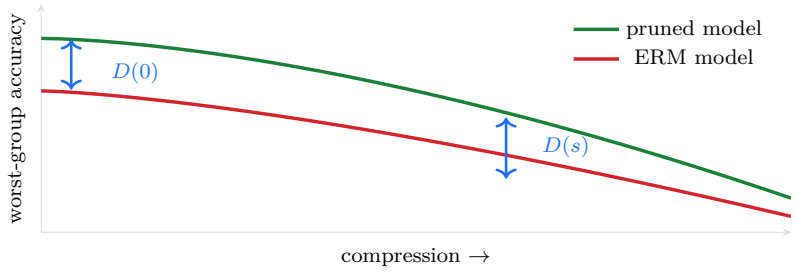
One more detail. The corrected rule also levels up the two bird classes, so that after deleting there are the same number of land birds as water birds. Waterbirds starts badly lopsided (3682 against 1113), and that lopsidedness is itself part of the problem. This is why the deletion budgets are so large – 25%, 50%, 75% – where the first attempt used 1–10%. Most of that budget is spent just evening up the two classes.

7.3 The measurement: retention ratio

Phase 2's headline number. Let W_p and W_e be worst-group accuracy of the **p**runed and **E**RM models at the same compression setting:

$$D(s) = W_p(s) - W_e(s) \qquad R(s) = \frac{D(s)}{D(0)}$$

In words: $D(0)$ is how much robustness the fix bought you before any compression. $D(s)$ is how much is left after compressing to level s. R is the fraction that survived. In the sketch below, the left arrow is $D(0)$ – the gain the fix bought you – and the right arrow is $D(s)$, however much of it is still there after compressing.



$R \approx 1$ – the two curves stay apart.
your robustness fix survives deployment

$R \approx 0$ – the curves merge.
compression silently undid your fix

7.4 The answers

Question one: does the fix work at all? Yes, clearly, and at every deletion budget we tried. All numbers are averages over 3 training runs.

model	worst group	average	gain in worst group
no fix (plain training)	73.3%	98.3%	—
delete 25%	86.4%	94.7%	**+13.1**
delete 50%	**87.9%**	94.4%	**+14.6**
delete 75%	87.3%	92.1%	**+14.0**

Two things to notice. The gain is roughly the same whichever budget you pick, which is reassuring – it means you don't have to tune the budget carefully. And the average accuracy *drops* by about 4 points while the worst group gains 14. That trade is the entire point: you are deliberately giving

up a little overall accuracy to stop the model relying on backgrounds. The paper reports the same shape of trade.

For reference, the paper gets 90.9% worst group where we get 87.9%. We are about 3 points short, on a different model and a tenth of the training time. Section 7.5 explains the gap honestly, because it is interesting in itself.

Question two – the real one: does the fix survive compression?

The answer, and it is not what we first thought

The fix survives compression right up to the point where compression has wrecked the model *for everybody*. Past that point the question stops meaning anything.

Here is magnitude pruning, the gentlest of the four methods:

weights removed	no fix	with fix	R	what it means
none	73.3%	87.9%	1.00	the fix bought 14.6 points
30%	58.4%	81.9%	**1.64**	still there, and wider than before
50%	23.7%	53.0%	**2.04**	still there
70%	13.7%	13.4%	−0.15	gone – both models are wrecked

Read the 70% row carefully, because it is easy to misread. R went to zero not because compression picked on the fix, but because at 70% sparsity *neither* model works any more – both are down near 13%. R is a ratio of two gaps, and when both models are broken there is no gap left to measure in either. A zero there means "question no longer applies", not "fix destroyed".

We checked this across all 195 settings that weren't collapsed, and it is the cleanest result in the project:

is the compressed model still usable?	settings	average R	share with $R \geq 0.75$
yes – worst group still above 40%	114	**+1.30**	**96%**
borderline – 25% to 40%	14	+1.30	86%
no – worst group below 25%	67	−0.20	13%

If compression leaves a working model, the fix is still there – 96% of the time, for all four compression methods. If compression has already destroyed the model, R is noise. There is no middle ground and no method that behaves specially.

The mistake we nearly published

Our first pass averaged R within each compression method and got numbers like "magnitude 1.26, FFN 0.31", and we wrote down that compression harms the fix differently depending on the method. That was wrong. Averaging R over compression levels smears a +2.04 **at 50% sparsity** together with a −0.15 **at 70%** and produces a number describing neither.

The methods do differ, but in *where the model dies*, not in whether the fix survives: FFN-neuron removal kills it by 30%, magnitude pruning survives to 50%, per-channel quantization survives all the way down to 3 bits. That is Phase 1's conclusion again – the four methods are not interchangeable.

Lesson for reading any of this: **R must always be read against a compression level, never averaged over them.**

7.5 Why we're 3 points short of the paper

Worth understanding, because it is a real limitation of the method rather than a bug in our code.

The fix depends on ranking training images by how hard they are. That ranking is only useful if the model hasn't learned the training set yet – once it gets nearly everything right, almost every image scores "easy" and the ranking stops carrying information.

The paper trains a ResNet-50 for 100 epochs and scores after the first. Ours is a pretrained vision transformer fine-tuned at a very low learning rate, and it reaches **97.9% training accuracy after two passes**. So by the time we score, the ranking has already gone flat:

of the images our rule deletes	chosen on real difficulty	already near-perfect
delete 25%	1080	**2017 (65%)**
delete 50%	1344	**2281 (63%)**
delete 75%	1479	**2676 (64%)**

About two-thirds of what we delete are images the model already classifies almost perfectly. The method's premise is "delete the hard ones", and we are mostly deleting easy ones, because there aren't enough hard ones left to fill the budget. What we're left doing is closer to "shrink and balance the training set", which is itself a known way to help the weak groups – hence we still get most of the benefit, just not all of it.

That's a genuine finding about the method: **it needs a model that hasn't yet fitted its training data.** In the transfer-learning setting most people actually work in, that condition is easy to violate without noticing.

7.6 Two things the code refuses to do

- **It won't divide by noise.** If $D(0)$ is tiny – meaning the data pruning didn't actually help on our setup – then $R = D(s)/D(0)$ would produce enormous meaningless ratios out of rounding error. So there's a gate: if the dense gain is below a threshold, R is left **undefined** and the result is reported as a null result. "It didn't work" is a real finding; a fabricated ratio is not.
- **It excludes collapsed models**, same as Phase 1. A dead model's "retention" is meaningless.

7.7 One non-obvious engineering bit

Scoring difficulty for seed 0 needs that seed's *epoch-1* weights. But seed 0 is already trained and its checkpoint is what produced all the Phase 1 results. Retraining it would overwrite that file – and training on Apple's GPU isn't bit-for-bit reproducible, so the new checkpoint wouldn't be identical. You'd silently decouple the published numbers from the weights behind them.

The fix is `-stop-after-epoch`: run the real schedule, stop early, write *no* checkpoint, keep only the scores. Costs 2 epochs instead of 10 and touches nothing.

8. How to actually read this code

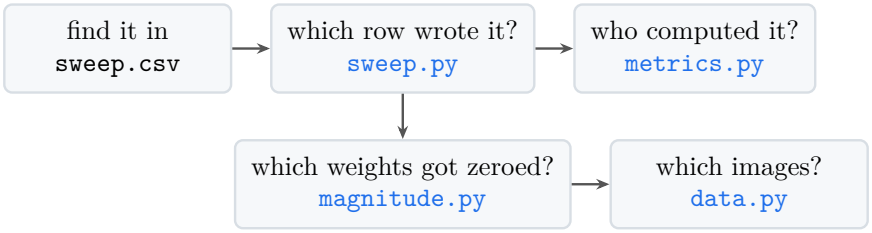
8.1 A reading order that works

1. `configs/base.yaml` – 2 minutes. Now you know every knob.
2. `src/metrics.py` – 10 minutes. Pure counting, no ML. Once `group_index` and `adjusted_average` click, the results table reads itself.
3. `src/evaluate.py` – 5 minutes. 50 lines. This is what "measuring the model" means here.
4. `src/compress/magnitude.py` – 15 minutes. The simplest method, and the tie-budget bit teaches you the mindset of the whole repo.

5. src/sweep.py – 20 minutes. Now you see how it all gets driven.
6. Everything else, as needed.

8.2 The exercise that makes it stick

Pick one number and chase it all the way through. Take **54.4%** – the worst-group accuracy after 30% magnitude pruning. Then:



Takes about half an hour and after it you'll know this codebase better than most people know code they wrote themselves.

8.3 Commands you'll actually use

Command	What it does
`pytest tests/ -q`	run all 52 gates (∼35s)
`python -m src.figures`	rebuild the three figures from the CSV
`python -m src.sweep -seed 0`	re-run the sweep (resumes if interrupted)
`./run_phase2.sh`	the whole Phase 2 pipeline
`caffeinate -i <cmd>`	stop the Mac sleeping mid-run

One practical gotcha

This machine is set to sleep after 1 minute idle. A sleeping Mac is why one training epoch in the Phase 1 log took **3338 seconds** when the others took **114**. Always prefix long runs with `caffeinate -i`, keep the lid open, and stay on mains power.

8.4 The three things to remember

1. **Average accuracy hides group damage.** 0.5 points vs 14.9 points, from the same operation.
2. **Structured compression hurts more than it looks.** 6 attention heads out of 72 cost 18 points of worst-group accuracy.
3. **A collapsed model can look fine.** A model that predicts one class for every image has no gap between its groups at all. Always check `pred_mean` before believing a small gap.
4. **The fix works, and it survives compression until compression breaks the model outright.** Worst group 73.3% → 87.9%, and that gain is still there 96% of the time in any setting where the compressed model is still usable.
5. **Never average a ratio over compression levels.** Doing that is how we briefly convinced ourselves of a conclusion that wasn't there.

Built from the repo as it stands. Both phases are run, over 3 seeds.
Phase 1 is in `results/sweep.csv`; Phase 2 in `results/sweep_spur*.csv` and `results/retention_*.csv`.
The first, failed Phase 2 attempt is kept in `old_run_Oct26/`.